UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Integração de Dados Legislativos e Inteligência Artificial para Consulta e Análise

Gustavo Jerônimo Moura de França

Orientador: Prof. Dr. Luiz Affonso H. Guedes de Oliveira

Natal, RN, 17 de janeiro de 2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Integração de Dados Legislativos e Inteligência Artificial para Consulta e Análise

Gustavo Jerônimo Moura de França

Orientador: Prof. Dr. Luiz Affonso H Guedes de Oliveira

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 17 de janeiro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

França, Gustavo Jerônimo Moura de.
Integração de Dados Legislativos e Inteligência Artificial para Consulta e Análise / Gustavo Jerônimo Moura de França. - 2025.
50f.: il.

Monografia (Graduação) - Universidade Federal do Rio Grande do Norte, Departamento de Engenharia de Computação e Automação, Engenharia de Computação, Natal, 2025.
Orientação: Dr. Luiz Affonso Henderson Guedes de Oliveira.

1. Dados legislativos - Monografia. 2. Inteligência artificial - Monografia. 3. Linguagem natural - Monografia. I. Oliveira, Luiz Affonso Henderson Guedes de. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira - CRB-15/429

Integração de Dados Legislativos e Inteligência Artificial para Consulta e Análise

Gustavo Jerônimo Moura de França

Monografia aprovada em 17 de janeiro, pela banca examinadora composta pelos seguintes membros:

Prof. Dr. Luiz Affonso H Guedes de Oliveira (orientador) . . . . . DCA/UFRN

Felipe Lima de Medeiros . . . . . PPGEEC/CT

Lemyson Oliveira Lemos . . . . . PPGEEC/CT

Agradecimentos

Quero agradecer a minha família que me proporcionou todo o suporte necessário para que eu pudesse desenvolver minha educação e me desenvolver como homem. A minha mãe, Leyla Larissa, meu pai, Robson Namara, que fizeram de tudo para me proporcionar a melhor educação, apoio, estrutura. A minha irmã Lanara Lourdes pelo apoio fornecido e por acreditar em mim. E aos meus cachorros, Luke e Rocky pela companhia nessa caminhada.

Agradeço a minha namorada, Maria Eduarda, que sempre está ao meu lado me fornecendo forças, suporte, ajudando com o possível e o impossível. Obrigado por ter me acompanhado do início ao fim dessa jornada e por ter me feito acreditar quando eu precisei.

Agradeço aos meus colegas de curso que me ajudaram de alguma forma nessa caminhada. Vocês foram importantes e me ajudaram diversas vezes, sem vocês talvez eu não conseguisse terminar o curso.

Agradeço aos meus amigos que acreditaram no meu potencial e me ajudaram a acreditar e tornar possível a realização desse trabalho, obrigado em especial a Loren por me ajudar nos últimos instantes.

Extendo a realização desse projeto ao meu eterno irmão de outra mãe, Pedro Henrique, vou sempre lembrar de você e da sua amizade, gratidão eterna por ter vivido uma amizade sincera contigo.

Agradeço a Deus e a Santa Rita por me dar forças.

Resumo

Este trabalho teve como objetivo desenvolver uma solução técnica, utilizando análise baseada em linguagem natural, mais especificamente de Modelos de Linguagem de Larga escala (Large Language model - LLMs), para facilitar o acesso aos dados legislativos da Câmara dos Deputados, com uma interface que proporciona uma experiência agradável, de fácil acesso que permite a execução de consultas de informações para análise. Para isso, foram coletados os dados em formato JSON disponibilizados pela Câmara dos Deputados e transformados em um banco de dados relacional MySQL.

O sistema faz o uso de uma LLM, integrada ao framework LangChain, que gera consultas SQL a partir de uma entrada de texto feita em linguagem natural, essa consulta é executada na base de dados e os resultados obtidos são apresentados em uma interface gráfica desenvolvida utilizando o framework Streamlit, que fornece ao usuário um campo específico para realizar perguntas e uma exibição das respostas obtidas.

Palavras-chave: Dados legislativos, inteligência artificial, linguagem natural, LangChain, Streamlit.

Abstract

This work aimed to develop a technical solution using natural language analysis, specifically Large Language Models (LLMs), to facilitate access to legislative data from the Brazilian Chamber of Deputies. The solution includes an interface designed to provide an intuitive and user-friendly experience, allowing users to execute queries for information analysis. To achieve this, JSON-format data provided by the Chamber of Deputies was collected and transformed into a relational MySQL database.

The system leverages an LLM integrated with the LangChain framework, which generates SQL queries based on natural language input. These queries are executed in the database, and the results are displayed in a graphical interface developed using the Streamlit framework. This interface offers users a dedicated input field for queries and presents the corresponding responses.

Keywords: Legislative data, artificial intelligence, natural language, LangChain, Streamlit.

Sumário

Sumário i

Lista de Figuras iii

Lista de Tabelas iv

Lista de Listings v

1 Introdução 1
- 1.1 Objetivos . . . 2
- 1.2 Estrutura do trabalho . . . 2

2 Fundamentação Teórica 3
- 2.1 Bancos de Dados Relacionais . . . 3
- 2.2 Inteligência Artificial . . . 5
  - 2.2.1 LLM . . . 6
  - 2.2.2 LangChain . . . 8
- 2.3 Interface Gráfica . . . 9

3 Implementação 11
- 3.1 Preparação dos Dados . . . 11
- 3.2 Modelagem e Construção do Banco de Dados . . . 14
- 3.3 Implementação do Sistema de Consultas . . . 17
- 3.4 Desenvolvimento da Interface do Usuário . . . 18
- 3.5 Testes e Validação . . . 20

4 Resultados 23
- 4.1 Interface . . . 23
- 4.2 Desempenho . . . 25
  - 4.2.1 Interpretação Contextual . . . 25
  - 4.2.2 Reconhecimento de Termos Ambíguos . . . 26
  - 4.2.3 Falsas Consultas . . . 28
- 4.3 Casos de Uso . . . 29

5 Conclusão 31

Referências Bibliográficas 32

A Informações adicionais **34**
A.1 Código em Python para preencher a tabela de proposições 34
A.2 Código em Python para preencher a tabela de Autores de Proposições . . 35
A.3 Código em Python para preencher a tabela de Temas de Proposições . . . 36
A.4 Código em Python para execução do sistema 36

Lista de Figuras

2.1 Exemplo de diagrama Entidade Relacionamento (ER). Fonte: www.edrawsoft.com. Disponível em: https://www.edrawsoft.com/pt/how-to-make-chen-er-diagram.html. Acesso em: Jan de 2025. 4
2.2 Diagrama do funcionamento. Fonte: Próprio Autor 6
2.3 Pontuações de Avaliação de Modelos. Fonte: www.openai.com. Disponível em: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/. Acesso em: Jan de 2025. 8
2.4 Diagrama de funcionamento da pipeline. Fonte: `www.langchain.com`. Disponível em: `https://python.langchain.com/docs/tutorials/sql_qa/`. Acesso em: Jan de 2025. 9
2.5 Exemplo de uma interface com Streamlit. Fonte: https://streamlit.io/. Disponível em: https://docs.streamlit.io/develop/tutorials/llms/llm-quickstart. Acesso em: Jan de 2025. 10

3.1 Interface em Streamlit 19
3.2 Resposta da Consulta Feita ao Banco de Dados. Fonte: Própio autor . . . 20
3.3 Figura 3.3: Consulta do autor com mais proposições. Fonte: Próprio autor 21
3.4 Consulta do autor com mais proposições. Fonte: Própio autor. 21
3.5 Consulta do autor com mais proposições. Fonte: Própio autor. 22
3.6 Consulta do autor com mais proposições. Fonte: Própio autor. 22

4.1 Visualização da interface em uma tela de notebook. Fonte: Própio autor. . 24
4.2 Visualização da interface em uma tela de dispositivo móvel. Fonte: Própio autor. 24
4.3 Resposta do sistema à uma pergunta "Quantas proposições relacionadas ao tema educação foram arquivadas em 2024?". Fonte: Própio autor. . . . 25
4.4 Resposta do sistema à uma pergunta com interpretação textual. Fonte: Própio autor. 26
4.5 Resposta do sistema à uma pergunta com termo ambíguo. Fonte: Própio autor. 27
4.6 Resposta do sistema a uma consulta errada. Fonte: Própio autor. 28
4.7 Resposta do sistema à uma pergunta. 30

Lista de Tabelas

4.1 Resultado da interaçãoo 26
4.2 Resultado da interação 27
4.3 Resultado da interação 29

Listings

3.1 Estrutura JSON de Proposições Legislativas 12
3.2 Estrutura JSON dos Autores das Proposições Legislativas 13
3.3 Estrutura JSON dos Temas das Proposições Legislativas 13
3.4 Criação da Tabela Proposicoes 14
3.5 Criação da Tabela do Último Status das Proposições 14
3.6 Criação da Tabela de Temas das Proposições 15
3.7 Criação da Tabela de Autores das Proposições 15
3.8 Carregamento e Inserção de Dados JSON no Banco de Dados MySql . . . 16
3.9 Função de Consulta a Base de Dados 17
3.10 Função de Consulta a Base de Dados 17
3.11 Função de Consulta a Base de Dados 18
3.12 Função de Consulta a Base de Dados 18
3.13 Função que gera a interface em Streamlit 19
A.1 Exemplo de código em Python 34
A.2 Exemplo de código em Python 35
A.3 Exemplo de código em Python 36
A.4 Exemplo de código em Python 36

Capítulo 1

Introdução

Nos últimos anos, a crescente digitalização das informações governamentais, com a chamada transformação digital, e a popularização de plataformas de dados abertos têm ampliado as possibilidades de análise e transparência legislativa. Tal fenômeno, vem possibilitando a criação de projetos Web que torna mais acessível o acesso aos dados à população.

Segundo o portal Dados Abertos da Câmara dos Deputados(CÂMARA DOS DEPUTADOS, 2025), em vários países do mundo, as pessoas estão cada vez mais interessadas e exigentes em relação à transparência no trabalho de seus governantes e das instituições e empresas públicas. Em resposta a esse desejo, as instituições onde esse trabalho acontece têm tornado suas informações cada vez mais públicas. No Brasil, por exemplo, a Lei de Acesso à Informação (LAI), de 2011, garante o direito de solicitar e receber informações de órgãos públicos, sem ter que pagar por isso, para qualquer cidadão ou entidade.

No Brasil, a Câmara dos Deputados disponibiliza uma vasta quantidade de informações públicas em formato digital. Entretanto, o formato de disponibilização desses dados é bastante técnico, sendo feito por meio de arquivos que não permitem uma fácil visualização como JSON e CSV. Contudo, apesar de serem arquivos de fácil acesso, percebe-se que ao mesmo tempo dificultam uma análise mais qualitativa e prática desses dados, sendo necessário um conhecimento mais aprofundado em ferramentas de manipulação de dados, para manipulá-los e assim realizar consultas e extrair informações que sejam úteis.

Neste contexto, é possível notar que existem sistemas Web que resolvem esse problema de acesso às informações, dentre eles, podemos citar: o projeto Akan, que facilita a criação de aplicações que utilizam dados públicos, disponível no Github, entre outros que estão disponíveis na comunidade do site dados abertos. Todavia, essas consultas ainda são limitadas e muito direcionadas ao contexto da aplicação.

Paralelamente, os avanços em inteligência artificial, principalmente com o advento recente dos Modelos de Linguagem de Larga Escala, abrem novas oportunidades para transformar a forma como interagimos com grandes volumes de dados. Ferramentas como o LangChain possibilitam integrar LLMs a bancos de dados, viabilizando a tradução de consultas em linguagem natural para comandos SQL precisos e eficientes(LANGCHAIN, 2025).

Com isso, é possível utilizar de Modelos de Linguagem de Larga Escala, como os apresentados por Brown et al. (2020) no GPT-3, pois permitem eliminar barreiras técni-

cas, tornando possível transformar consultas textuais em interações com sistemas computacionais de maneira direta e eficiente, possibilitando uma interação muito intuitiva e abrangente entre os usuários e as informações.

Desse modo, o presente trabalho tem como objetivo desenvolver uma solução que integre dados legislativos da Câmara dos Deputados a uma plataforma de inteligência artificial capaz de realizar consultas e análises de maneira intuitiva e acessível. A proposta abrange três etapas principais: (i) extração e transformação de dados legislativos em JSON para um banco de dados relacional MySQL; (ii) utilização de uma LLM utilizando LangChain, que gera consultas SQL a partir de perguntas feitas por usuários; e (iii) desenvolvimento de uma interface interativa com Streamlit, permitindo que qualquer pessoa, independentemente do seu conhecimento técnico, possa explorar e analisar os dados legislativos.

Assim, ao conectar os domínios de ciência de dados, inteligência artificial e transparência governamental, este trabalho visa facilitar o acesso a informações legislativas e promover o uso consciente dos dados públicos. Além disso, busca-se demonstrar o potencial de tecnologias modernas na simplificação de processos complexos e no fortalecimento da cidadania ativa.

1.1 Objetivos

A principal meta deste trabalho é desenvolver o projeto de um sistema que realmente facilite a consulta das pessoas aos dados da Câmara dos Deputados. De modo que esse acesso seja mais democrático e prático, tanto para a população em geral, como pessoas da própria área política que desejam fazer consultas ou pesquisas, utilizando estes dados.

1.2 Estrutura do trabalho

Este trabalho está organizado em 5 capítulos, com o primeiro dedicado à introdução. O capítulo 2 com foco na teoria utilizada para desenvolver o projeto, assim como as tecnologias utilizadas, como elas funcionam e o motivo de terem sido escolhidas para esse projeto. O capítulo 3 tem como foco apresentar a forma como o projeto foi pensada seguida pela implementação do funcionamento do sistema. O capítulo 4 é feito uma análise dos resultados obtidos através do funcionamento do sistema e se ele cumpre a ideia que foi proposta. Por conseguinte o capítulo 5 irá concluir a ideia do projeto, abordando os pontos positivos do sistema, assim como futuras melhorias e possíveis formas de implantação do projeto.

Capítulo 2

Fundamentação Teórica

Neste capítulo, serão apresentados os fundamentos teóricos necessários para a implementação do projeto. Será abordado com mais clareza as tecnologias utilizadas para o desenvolvimento, as ferramentas necessárias e os conceitos utilizados no desenvolvimento da solução. Os motivos que levaram às decisões feitas para escolha dessas tecnologias também serão explicadas e analisadas.

Inicialmente, é abordada a criação da base relacional e os dados utilizados para a criação da mesma, em seguida é abordado os modelos de linguagem natural, os motivos que levaram a escolha da LLM utilizada no projeto, as ferramentas que auxiliaram o seu uso, e por fim a biblioteca utilizada para a criação da interface gráfica que irá permitir o uso do sistema.

2.1 Bancos de Dados Relacionais

Bancos de dados relacionais são um dos métodos mais consolidados e amplamente utilizados para o armazenamento e gerenciamento de informações. Baseados no modelo relacional proposto por E. F. Codd na década de 1970, esses bancos organizam os dados em tabelas compostas por linhas e colunas, permitindo a criação de relações entre as tabelas por meio de chaves primárias e estrangeiras (Codd, 1970). Essa abordagem oferece vantagens como consistência, integridade referencial e suporte para consultas complexas utilizando a linguagem SQL, amplamente adotada na indústria e academia (Elmasri & Navathe, 2016).

Para o desenvolvimento deste projeto, foi escolhido um sistema de gerenciamento de banco de dados, para que as informações coletadas a partir de arquivos JSON pudessem ser transformados em conjuntos de dados estruturados e de fácil acesso, o MySql foi escolhido por proporcionar algumas vantagens em relação a outros gerenciadores de banco de dados, sendo algum dos motivos:

- **Popularidade e Comunidade Ativa**: O MySQL é um dos SGBDs mais utilizados no mundo, contando com ampla documentação, suporte comunitário e uma vasta gama de bibliotecas e ferramentas.
- **Desempenho e Eficiência**: O MySQL oferece alto desempenho para operações de leitura e escrita, sendo adequado para lidar com grandes volumes de dados, como os obtidos dos arquivos JSON legislativos.

- **Facilidade de Integração**: O MySQL possui excelente compatibilidade com ferramentas de desenvolvimento, incluindo bibliotecas Python, como `mysql-connector` e `SQLAlchemy`, utilizadas neste trabalho para conexão e manipulação de dados.

O uso do MySQL permitiu a integração direta com o sistema de inteligência artificial desenvolvido, uma vez que a consulta gerada em linguagem SQL, pelo modelo de linguagem natural (LLM), pode ser executada diretamente no banco, retornando os resultados de forma rápida e precisa. Essa característica foi essencial para o sucesso da interface de consulta em linguagem natural desenvolvida com Streamlit, que depende de respostas imediatas para oferecer uma experiência de usuário satisfatória.

Com a manipulação dos dados, foi necessário, para desenvolvimento do banco de dados, a modelagem de entidades de acordo com a estrutura dos dados obtidos, sendo importante modelar os objetos JSON em uma estrutura de entidade relacionamento que são interligadas e que permitem criar uma entendimento e separação melhor das informações, como demonstrado na figura 2.1.

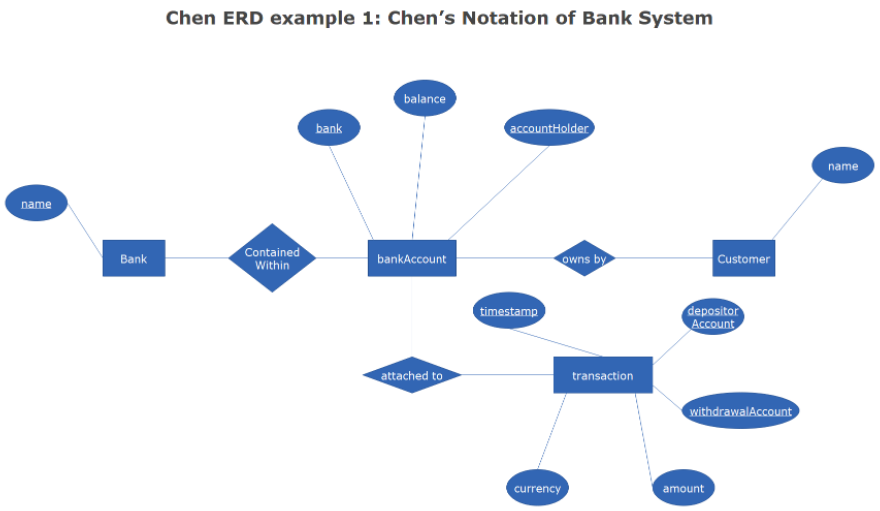


Figura 2.1: Exemplo de diagrama Entidade Relacionamento (ER). Fonte: www.edrawsoft.com. Disponível em: https://www.edrawsoft.com/pt/how-to-make-chen-er-diagram.html. Acesso em: Jan de 2025.

Os modelos de ER são fundamentais no design de bancos de dados, pois permitem a representação abstrata das estruturas de dados e seus relacionamentos. Esse modelo, introduzido por Peter Chen em 1976, é amplamente utilizado para o planejamento e documentação de sistemas relacionais. Nele, os dados são representados por entidades, que podem ser objetos ou conceitos relevantes ao contexto do sistema, enquanto os relacionamentos descrevem como essas entidades interagem entre si.

Além disso, atributos são associados às entidades e aos relacionamentos para descrever suas características (CHEN, 1976). O modelo ER é uma ferramenta essencial para

garantir a integridade dos dados, a eliminação de redundâncias e o alinhamento entre os requisitos do sistema e a estrutura do banco de dados (ELMASRI; NAVATHE, 2016). Durante o processo de implementação, o modelo ER é convertido em esquemas de tabelas relacionais, sendo esta uma etapa crucial para a criação de bancos de dados consistentes e eficientes.

2.2 Inteligência Artificial

A Inteligência Artificial (IA) pode ser definida como a área da ciência da computação focada no desenvolvimento de sistemas capazes de realizar tarefas que, normalmente, exigiriam inteligência humana. Essas tarefas incluem percepção, raciocínio, tomada de decisão e até mesmo a comunicação em linguagem natural. A IA é uma das tecnologias mais transformadoras da atualidade e tem sido aplicada em diversos setores, incluindo saúde, finanças, educação e, mais recentemente, em processos legislativos e análise de dados (Russell & Norvig, 2021).

O Processamento de Linguagem Natural é um subcampo da área de computação e da inteligência artificial, que tem como foco as interações entre linguagem humana e computadores. O PLN engloba tarefas como tradução automática, resumo de texto, análise de sentimentos e, no contexto de sistemas de consulta, a conversão de linguagem natural em comandos que um computador pode executar, como a transformação de uma consulta em linguagem natural em uma query SQL. Conforme Manning e Schütze (1999), "o processamento de linguagem natural permite que os computadores compreendam e respondam adequadamente às solicitações feitas em linguagem humana".

Esse processo de tradução de texto livre para uma linguagem de consulta estruturada envolve uma série de tarefas de PLN, como:

1. **Análise Sintática:** Identificar as partes da pergunta e suas relações. Por exemplo, em uma pergunta como: "Quais foram as proposições legislativas aprovadas em 2024?", o sistema precisa entender que o ano é um parâmetro e que "proposições legislativas" é o sujeito da consulta.
2. **Extração de Entidades:** Determinar os elementos chave da pergunta, como datas, autores e tipos de proposições. Essas entidades são usadas para construir a consulta SQL.
3. **Contextualização da Pergunta:** Através do uso de prompts e modelos de linguagem, o sistema é capaz de identificar o contexto da consulta e gerar uma resposta precisa com base na estrutura do banco de dados. Por exemplo, ele pode discernir que uma "proposição legislativa"está associada a tabelas de "proposições" e "votações".

Para o funcionamento do sistema é necessário criar uma cadeia de execuções interligadas que o resultado de uma seja a entrada da outra, pois dessa forma é possível criar uma pipeline de execuções que responsabiliza cada etapa em um determinado escopo, permitindo assim executar qualquer ação necessária entre os estados, como ilustrado na figura 2.2.

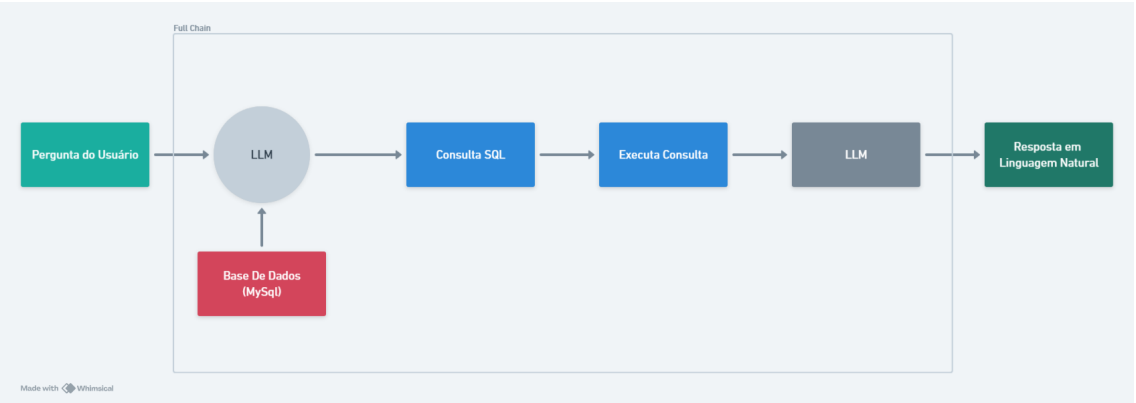


Figura 2.2: Diagrama do funcionamento. Fonte: Próprio Autor

O funcionamento principal deste sistema envolve a conversão de perguntas em linguagem natural, formuladas pelos usuários, em consultas SQL executáveis no banco de dados. A chave para essa transformação é utilizar um modelo de linguagem avançado, que seja capaz de compreender a intenção por trás de uma pergunta e gerar a consulta SQL apropriada. O modelo tem a função de interpretar o texto de entrada, reconhecer a consulta implícita e converter essa consulta para uma linguagem estruturada, como o SQL.

Após a execução da consulta SQL, o sistema além de coletar a resposta tem que analisar e extrair as informações necessárias para exibi-las de forma estruturada, seguindo uma linguagem amigável.

Além disso, o PLN torna possível que o sistema compreenda uma vasta gama de perguntas, mesmo quando formuladas de maneira imprecisa ou não estruturada. A flexibilidade do PLN também permite que o sistema evolua e se adapte a novas consultas, tornando-o mais robusto e eficaz ao longo do tempo.

2.2.1 LLM

Os modelos de linguagem de larga escala são redes neurais treinadas em grandes volumes de dados textuais, capazes de gerar e compreender linguagem natural em diversos contextos(Bommasani et al., 2021, p. 5). Essa tecnologia é o símbolo de um dos principais avanços na área de Inteligência Artificial, com uma capacidade extraordinária de processar e gerar linguagem natural com resultados fantásticos.

Essa tecnologia se destaca por sua utilidade ao ser aplicada a problemas específicos do dia a dia, especialmente aqueles que ainda não possuem uma solução técnica previamente desenvolvida. Entre suas principais funcionalidades, destaca-se a capacidade da inteligência artificial de auxiliar na criação de novas soluções e no desenvolvimento de sistemas mais robustos. Além disso, a IA pode ser integrada a um chat, funcionando como um assistente, proporcionando suporte para esclarecer dúvidas e consultar informações de forma eficiente.

Com a crescente popularidade dessa tecnologia, é possível perceber também o aumento na quantidade de modelos que estão disponíveis para uso, desde os mais bási-

cos aos mais avançados, que possuem otimizações. LLMs como GPT-3 representam um avanço significativo, oferecendo capacidades robustas para resolver tarefas de linguagem natural com instruções mínimas, devido ao treinamento em escalas massivas de dados(Brown et al., 2020, p. 7).

Nesse sentido, para escolher o modelo é possível encontrar algumas nuances entre eles, que podem ser decisivas no processo de escolha. De acordo com a Meta, "o Llama é uma solução mais leve e eficiente para tarefas de Processamento de Linguagem Natural (PLN), mantendo alta capacidade de generalização e desempenho em uma variedade de tarefas"(META, 2023).

A escolha do GPT-4 como LLM para este projeto se deve à sua alta capacidade de compreensão e geração de linguagem natural, aliada à flexibilidade para interpretar consultas complexas e adaptá-las ao contexto. Sua eficiência em transformar texto em consultas SQL precisas o torna ideal para integrar dados legislativos e fornecer respostas relevantes de maneira rápida e confiável.

Nesse cenário, o modelo GPT-4o Mini parece ser uma interessante opção. Segundo a OpenAI, o GPT-4o Mini é uma versão compacta do GPT-4o, projetada para oferecer capacidades avançadas de processamento de linguagem natural com menor custo e maior eficiência computacional, mantendo um desempenho superior em benchmarks acadêmicos (OPENAI, 2024). Os gráficos benchmark apresentados na figura 2.4 reforçam essas característica do modelo e destacam os resultados do mesmo em comparação com outras opções.

De forma adicional, os prompts possuem um papel importante no uso e na eficiência dos modelos de linguagem de larga escala, como GPT e similares. Um prompt pode ser a entrada textual fornecida ao modelo, projetada para orientar sua resposta de forma precisa e contextualizada. A qualidade e a clareza do prompt são determinantes para a geração de resultados relevantes, especialmente em tarefas complexas como criação de textos, tradução, análise de dados e geração de código.

Prompts bem elaborados ajudam o modelo a entender exatamente o que o usuário deseja, maximizando sua capacidade de fornecer respostas úteis. Segundo Liu et al. (2023), "os prompts servem como uma interface entre o usuário e o modelo, sendo fundamentais para explorar plenamente o potencial de grandes modelos de linguagem." A capacidade de construir prompts eficientes é, portanto, uma habilidade essencial para aproveitar o poder dessas tecnologias.

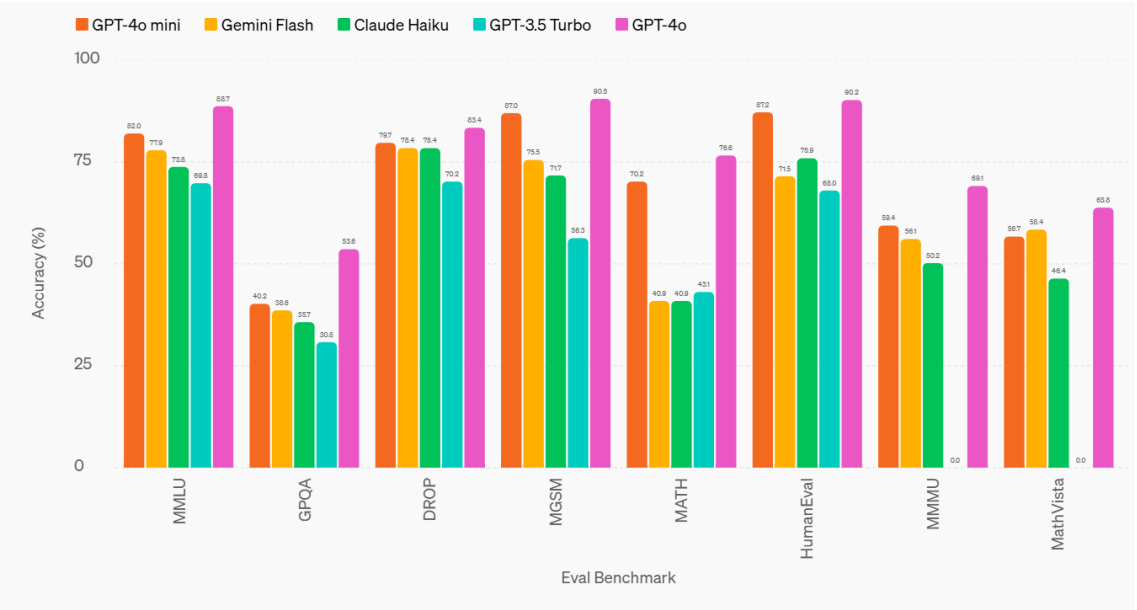


Figura 2.3: Pontuações de Avaliação de Modelos. Fonte: www.openai.com. Disponível em: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/. Acesso em: Jan de 2025.

2.2.2 LangChain

O LangChain é uma estrutura de código aberto que facilita o desenvolvimento de aplicativos impulsionados por grandes modelos de linguagem (LLMs). Conforme descrito em sua documentação oficial, o LangChain "simplifica todas as etapas do ciclo de vida de aplicativos baseados em LLMs. Isso inclui desde a integração com fontes de dados externas até a orquestração de fluxos de trabalho complexos, permitindo que desenvolvedores criem soluções personalizadas de forma mais eficiente. (LANGCHAIN, 2025).

De acordo com Harrison (2023), "o LangChain possibilita a construção de aplicações dinâmicas, permitindo que grandes modelos de linguagem sejam combinados com dados externos e ações personalizadas para resolver problemas específicos." Sua flexibilidade o torna ideal para aplicações como geração de consultas SQL, assistentes virtuais e análise de grandes volumes de dados.

O núcleo funcional deste projeto foi desenvolvido utilizando uma cadeia de execuções do LangChain, que conecta diretamente o usuário, o modelo de linguagem, e o banco de dados relacional (MySQL). O processo de funcionamento do modelo de linguagem é simples, mas muito poderoso, pois é possível ser coordenado por uma sequência de execuções que geram o resultado esperado, conforme mostrado na figura 2.3.

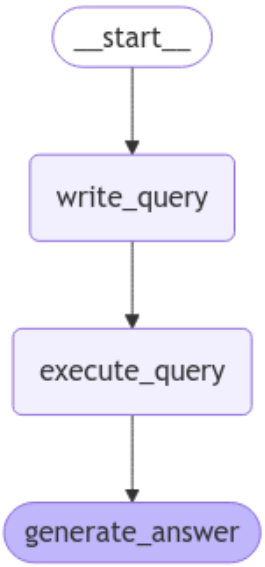


Figura 2.4: Diagrama de funcionamento da pipeline. Fonte: `www.langchain.com`. Disponível em: `https://python.langchain.com/docs/tutorials/sql_qa/`. Acesso em: Jan de 2025.

A integração entre LangChain, LLMs e o banco de dados é essencial para atingir o objetivo principal deste trabalho: oferecer uma interface acessível que permita consultas complexas de maneira intuitiva. Essa abordagem não apenas simplifica a interação com os dados legislativos, mas também demonstra o potencial de tecnologias modernas em resolver desafios relacionados ao acesso e análise de grandes volumes de informações.

2.3 Interface Gráfica

As interfaces gráficas possuem um papel fundamental para proporcionar uma boa experiência no uso de sistemas computacionais, possibilitando uma conexão entre o computador e uma pessoa, que fornece dados de entrada para alguma operação ser executada, sendo acessível por meios visuais e de maneira intuitiva. Uma boa interface gráfica deve priorizar a usabilidade e a experiência do usuário, proporcionando clareza, eficiência e satisfação durante a interação (Shneiderman et al. 2016).

Então, para criação de uma interface, ela deve ser simples, acessível e que possibilite a conexão entre o usuário e o sistema. Existem várias soluções para essa situação, mas uma em especial possui uma integração muito prática com o contexto do projeto e a sua aplicação, o Streamlit. Essa referida biblioteca transforma scripts de dados em aplicações compartilháveis, permitindo que desenvolvedores concentrem-se nos dados e na lógica, em vez de se preocuparem com a infraestrutura(STREAMLIT, 2025).

O Streamlit é uma ferramenta de código aberto para a criação de aplicativos Web interativos e dinâmicos, ele possui um foco em simplicidade e facilidade na experiência de uso. Com o Streamlit, é possível criar interfaces gráficas para aplicações em Python de forma rápida. A principal vantagem do Streamlit é a sua capacidade de transformar

scripts Python simples em aplicações web totalmente funcionais, sem a necessidade de frameworks complexos ou configurações adicionais.

O uso dessa ferramenta é altamente acessível, permitindo que desenvolvedores criem interfaces com apenas algumas linhas de código. Ele oferece uma API simples que facilita a criação de componentes como formulários, gráficos, tabelas, e botões. Sendo uma ferramenta construída para Python, o Streamlit integra-se facilmente com bibliotecas como Pandas, NumPy, Matplotlib, Plotly, entre outras. Isso permite a criação de interfaces de visualização de dados e manipulação de informações sem a necessidade de trabalhar com JavaScript ou HTML.

No contexto desse projeto, o Streamlit foi utilizado para desenvolver a interface do sistema, ilustrado na figura 2.5, permitindo que os usuários interajam com a funcionalidade de consulta de dados legislativos de forma intuitiva e direta. Através da interface criada, os usuários podem inserir perguntas em linguagem natural e visualizar as respostas geradas automaticamente a partir das informações do banco de dados.

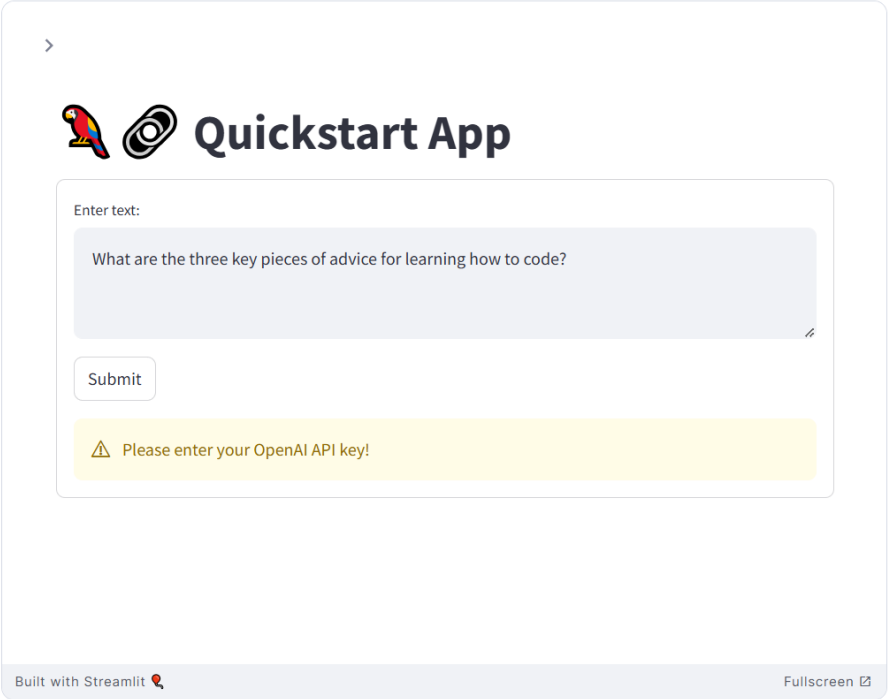


Figura 2.5: Exemplo de uma interface com Streamlit. Fonte: https://streamlit.io/. Disponível em: https://docs.streamlit.io/develop/tutorials/llms/llm-quickstart. Acesso em: Jan de 2025.

Capítulo 3

Implementação

O objetivo desta seção é descrever os métodos e ferramentas utilizadas para o desenvolvimento de um sistema de consulta e análise de dados legislativos, utilizando conhecimentos de ciência de dados para obtenção e manipulação de dados, banco de dados para um melhor armazenamento e utilização dos dados, Inteligência Artificial para Processamento de Linguagem Natural e uma interface interativa para melhorar e facilitar a experiência de acesso aos dados.

A etapa de implementação deste projeto representa a parte prática das metodologias. O desenvolvimento foi feito de forma modular e por etapas, o que permitiu uma melhor separação de cada parte de forma independente, mas complementares, como a primeira parte de manipulação dos dados que teve como resultado a criação do banco de dados estruturado, passando pela parte do LangChain que faz consultas a essa base de dados e por último a interface gráfica que permite uma interação fácil.

3.1 Preparação dos Dados

O acesso a informações públicas em formatos abertos é um dos pilares da transparência governamental e da promoção de uma cidadania ativa. No Brasil, o PDA desempenha um papel central nesse processo. Segundo o Governo Federal, "o Plano de Dados Abertos (PDA) é o instrumento que operacionaliza a Política de Dados Abertos do Poder Executivo Federal, pois planeja as ações que visam a abertura e sustentação de dados abertos nas organizações públicas. Cada órgão/entidade possui a obrigação de elaborar um PDA com vigência de dois anos, a contar da publicação do documento"(BRASIL, 2023).

No âmbito do Poder Legislativo, a Câmara dos Deputados também implementa ações alinhadas ao conceito de dados abertos, consolidando seu próprio PDA. Por meio do portal Dados Abertos da Câmara dos Deputados, a instituição disponibiliza informações detalhadas sobre proposições legislativas, votações, presenças parlamentares, despesas e outras atividades. Esses dados estão disponíveis em formatos como JSON, XML e CSV, garantindo que possam ser reutilizados e redistribuídos por qualquer pessoa.

A existência de um PDA na Câmara dos Deputados é fundamental para fortalecer a transparência e fomentar a participação cidadã. A partir desses dados, é possível realizar análises detalhadas sobre a atuação parlamentar, identificar padrões de comportamento e desenvolver soluções tecnológicas que democratizam o acesso a informações legislativas.

No presente trabalho, a integração entre os dados abertos da Câmara e ferramentas de inteligência artificial exemplifica como o uso desses recursos pode transformar a maneira como cidadãos e instituições interagem com informações públicas.

Coleta de Dados Legislativos

Para a coleta dos dados foi utilizada a plataforma dos dados abertos da câmara federal, que disponibiliza informações sobre as proposições legislativas em arquivos de formato JSON separados por ano, para o contexto do projeto foram utilizados os últimos 5 anos de dados disponíveis, de 2020 até 2024.

Estrutura dos Arquivos JSON

Os arquivos JSON coletados são separados em 3 tipos, apresentando uma estrutura hierárquica e bastante relacional, o código 3.1 contém informações mais cruciais sobre as proposições legislativas, como o ano, a ementa, o tipo, o seu identificador. Também possui um objeto responsável apenas para informar o status mais atual da mesma.

```
{
  "id": number,
  "uri": string,
  "siglaTipo": string,
  "numero": number,
  "ano": number,
  "codTipo": number,
  "descricaoTipo": string,
  "ementa": string,
  "ementaDetalhada": string,
  "keywords": string,
  "dataApresentacao": string,
  "uriOrgaoNumerador": string,
  "uriPropAnterior": string,
  "uriPropPrincipal": string,
  "uriPropPosterior": string,
  "urlInteiroTeor": string,
  "ultimoStatus": {
    "data": string,
    "sequencia": string,
    "uriRelator": string,
    "codOrgao": string,
    "siglaOrgao": string,
    "uriOrgao": string,
    "regime": string,
    "descricaoTramitacao": string,
    "idTipoTramitacao": string,
    "descricaoSituacao": string,
    "idSituacao": string,
```

```
30      "despacho": string,
31      "apreciacao": string,
32      "url": string
33    }
34  }
```

Código 3.1: Estrutura JSON de Proposições Legislativas

O código 3.2 já contém dados focados nos autores das proposições, possuindo uma dependência direta delas, já que guarda a informação do identificador das proposições, que mais tarde irá ser usado como chave estrangeira no banco de dados.

```
1  {
2        "idProposicao": number,
3        "uriProposicao": string,
4        "uriAutor": string,
5        "codTipoAutor": number,
6        "tipoAutor": string,
7        "nomeAutor": string,
8        "siglaPartidoAutor": string,
9        "uriPartidoAutor": string,
10       "siglaUFAutor": string,
11       "ordemAssinatura": string,
12       "proponente": string
13 }
```

Código 3.2: Estrutura JSON dos Autores das Proposições Legislativas

Por conseguinte, o código 3.3 exemplifica uma estrutura menor, que é responsável por armazenar informações dos temas das proposições legislativas, assim como no código anterior, existe uma dependência dos dados de proposições, que será utilizada para criação das entidades no banco de dados.

```
1  {
2        "uriProposicao": string,
3        "siglaTipo": string,
4        "numero": number,
5        "ano": number,
6        "codTema": number,
7        "tema": string,
8        "relevancia": number
9  }
```

Código 3.3: Estrutura JSON dos Temas das Proposições Legislativas

Essas estruturas revelam informações sobre as proposições, mas sobre pontos diferentes, que refletem perfeitamente uma estrutura de banco de dados relacionais, a partir disso foi possível separar melhor os conjuntos em entidades no banco de dados que se conectam e podem ser relacionadas.

3.2 Modelagem e Construção do Banco de Dados

Após a análise da estrutura das proposições nos arquivos JSON, foi possível criar uma lógica que percorre esses dados e transforma as informações nos campos em colunas e entidades para a base de dados criada em MySql, com ajuda de um script Python foi possível iniciar uma conexão local com o banco e fazer a criação das tabelas com SQL, exemplificado no código 3.4.

A modelagem do banco de dados seguiu os princípios de normalização, organização das informações em tabelas inter-relacionadas para evitar redundância e permitir consultas eficientes. A divisão dessas tabelas também permitiu uma separação e melhor gerenciamento dos dados.

Na código 3.4, é possível observar a criação da entidade `Proposicoes` que contém as informações principais sobre as proposições legislativas, ela é a entidade central da estrutura.

```
CREATE TABLE Proposicoes (
    id INT PRIMARY KEY,
    uri VARCHAR(255),
    siglaTipo VARCHAR(10),
    numero INT,
    ano INT,
    codTipo INT,
    descricaoTipo VARCHAR(255),
    ementa TEXT,
    ementaDetalhada TEXT,
    keywords TEXT,
    dataApresentacao DATETIME,
    uriOrgaoNumerador VARCHAR(255),
    uriPropAnterior VARCHAR(255),
    uriPropPrincipal VARCHAR(255),
    uriPropPosterior VARCHAR(255),
    urlInteiroTeor VARCHAR(255)
);
```

Código 3.4: Criação da Tabela Proposicoes

A entidade `UltimoStatus` possui os dados da última atualização registrada de uma determinada proposição. Na código 3.6, é possível observar a criação desta entidade e sua estrutura, que utiliza o identificador único da entidade `Proposicoes` como chave estrangeira.

```
CREATE TABLE UltimoStatus (
    id INT PRIMARY KEY AUTO_INCREMENT,
    data DATE,
    sequencia VARCHAR(255),
    uriRelator VARCHAR(255),
    codOrgao VARCHAR(255),
    siglaOrgao VARCHAR(10),
```

```
8       uriOrgao VARCHAR(255),
9       regime VARCHAR(255),
10      descricaoTramitacao VARCHAR(255),
11      idTipoTramitacao INT,
12      descricaoSituacao VARCHAR(255),
13      idSituacao VARCHAR(255),
14      despacho TEXT,
15      apreciacao VARCHAR(255),
16      url VARCHAR(255),
17      proposicao_id INT,
18      FOREIGN KEY (proposicao_id) REFERENCES Proposicoes(id)
19  );
```

Código 3.5: Criação da Tabela do Último Status das Proposições

Para armazenar os dados relacionados aos temas das proposições, foi criada a tabela `ProposicoesTemas`, mostrada na código 3.6, que possui uma estrutura simples e se relacionando diretamente com a tabela `Proposicoes`.

```
1   CREATE TABLE ProposicoesTemas (
2       uriProposicao VARCHAR(255) NOT NULL,
3       siglaTipo VARCHAR(10) NOT NULL,
4       numero INT NOT NULL,
5       ano INT NOT NULL,
6       codTema INT NOT NULL,
7       tema VARCHAR(255) NOT NULL,
8       relevancia TINYINT NOT NULL,
9       PRIMARY KEY (uriProposicao, codTema)
10  );
```

Código 3.6: Criação da Tabela de Temas das Proposições

Para concluir a transformação dos dados dos arquivos para a estrutura do banco de dados, foi criada a tabela `ProposicoesAutores`, como demonstra a código 3.7, que armazena os dados relacionados aos autores das proposições.

```
1   CREATE TABLE ProposicoesAutores (
2       idProposicao INT NOT NULL,
3       uriProposicao VARCHAR(255) NOT NULL,
4       uriAutor VARCHAR(255) NOT NULL,
5       codTipoAutor INT NOT NULL,
6       tipoAutor VARCHAR(100) NOT NULL,
7       nomeAutor VARCHAR(255) NOT NULL,
8       siglaPartidoAutor VARCHAR(50),
9       uriPartidoAutor VARCHAR(255),
10      siglaUFAutor VARCHAR(10),
11      ordemAssinatura INT NOT NULL,
12      proponente BOOLEAN NOT NULL,
13      PRIMARY KEY (idProposicao, uriAutor)
14  );
```

Código 3.7: Criação da Tabela de Autores das Proposições

Para preencher as tabelas criadas com os dados dos arquivos, foi necessário desenvolver uma lógica iterativa para percorrer todos os elementos dos arquivos, coletar essas informações e salvar na base de dados, para isso foi utilizado um código em Python, que permite uma fácil manipulação dos dados do arquivo. Para conexão com o MySql foi utilizada uma biblioteca MySQL Connector do Python, que permitiu criar uma conexão com a base de dados local e executar comandos SQL.

O código 3.8 demonstra como foi feito o carregamento do arquivo JSON contendo proposições e a inserção das informações na tabela `Proposicoes` do banco de dados.

```
# Caminho do arquivo JSON
file_path = 'proposicoes.json'  # Substitua 'file_path' pelo caminho correto

# Abrir o arquivo JSON e carregar os dados
with open(file_path, 'r') as file:
    proposicoes_data = json.load(file)

# Iterar sobre as proposicoes no JSON e inserir no banco de dados
for proposicao in proposicoes_data["dados"]:
    cursor.execute("""
        INSERT INTO Proposicoes (
            id, uri, siglaTipo, numero, ano, codTipo, descricaoTipo, ementa,
            dataApresentacao
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
    """, (
        proposicao["id"], proposicao["uri"], proposicao["siglaTipo"],
        proposicao["numero"], proposicao["ano"], proposicao["codTipo"],
        proposicao["descricaoTipo"], proposicao["ementa"],
        proposicao["dataApresentacao"]
    ))
```

Código 3.8: Carregamento e Inserção de Dados JSON no Banco de Dados MySql

Para reaproveitamento do código anterior, é preciso apenas alterar o arquivo que deseja utilizar, alterando o `file_path` e atualizar a consulta SQL utilizada para se adaptar a estrutura da tabela, assim como os dados que vão ser utilizados por cada arquivo.

3.3 Implementação do Sistema de Consultas

A principal funcionalidade do projeto, que é a conversão de uma consulta em linguagem natural em uma consulta SQL, foi implementada utilizando a biblioteca LangChain para integrar um Modelo de Linguagem de Larga Escala e gerar consultas SQL.

Com o banco estabelecido, é possível utilizar os dados estruturados para fazer consultas utilizando a LLM da OpenAI e o sistema de pipeline do LangChain, que permite estruturar uma cadeia de execuções separadas, de forma que ao serem executadas geram em etapas. Nisso, temos primeiro uma consulta SQL com base na entrada de texto, em seguida essa consulta é executada na base de dados que retorna informações que serão transformadas pela LLM em uma resposta amigável e estruturada será gerada em linguagem natural.

O código ficou organizado em 3 etapas, que geram a resposta de acordo com a entrada do usuário, mas para a execução dessas etapas foi necessário criar o sistema de execuções, como mostrado no código 3.9, utilizando o LangGraph, que é uma biblioteca de construções de aplicações com estado e múltiplos agentes utilizando LLMs, utilizada para criar fluxos de trabalho de agentes e multiagentes(LANGCHAIN, 2025).

```
graph_builder = StateGraph(State).add_sequence(
    [write_query, execute_query, generate_answer]
)

graph_builder.add_edge(START, "write_query")
```

Código 3.9: Função de Consulta a Base de Dados

Para a primeira etapa, o código 3.10 demonstra o código que gera a consulta SQL a ser executada. Nessa etapa são passadas em formato de prompt, informações sobre a base de dados, como a estrutura do banco de dados, o limite para geração da resposta e a própria entrada do usuário, que será transformada pela LLM.

```
def write_query(state: State):
    prompt = query_prompt_template.invoke(
        {
            "dialect": db.dialect,
            "top_k": 1000,
            "table_info": db.get_table_info(),
            "input": state["question"],
        }
    )
    structured_llm = llm.with_structured_output(QueryOutput)
    result = structured_llm.invoke(prompt)
    return {"query": result["query"]}
```

Código 3.10: Função de Consulta a Base de Dados

Na próxima etapa, é feita a execução da consulta, conforme demonstrado no código 3.11, que foi repassada, ela tem apenas a responsabilidade de executar a query na base de dados e repassar essa informação para a próxima etapa.

```
def execute_query(state: State):
    execute_query_tool = QuerySQLDataBaseTool(db=db)
    return {"result": execute_query_tool.invoke(state["query"])}
```

Código 3.11: Função de Consulta a Base de Dados

Na última etapa de execução é recebido todo o resultado da consulta na base de dados. Nisso, essa consulta será repassada novamente para a LLM, mas agora com o intuito de resumir os dados em uma resposta organizada para o usuário, demonstrado no código 3.12.

```
def generate_answer(state: State):
    prompt = (
        "Given the following user question, corresponding SQL
           query, "
        "and SQL result, answer the user question.\n\n"
        f'Question: {state["question"]}\n'
        f'SQL Query: {state["query"]}\n'
        f'SQL Result: {state["result"]}'
    )
    response = llm.invoke(prompt)
    return {"answer": response.content}
```

Código 3.12: Função de Consulta a Base de Dados

3.4 Desenvolvimento da Interface do Usuário

Para desenvolvimento da interface de acesso ao sistema, a proposta é utilizar uma biblioteca prática que possibilitasse integração fácil com código Python, então a escolha foi o Streamlit, que possibilita o desenvolvimento simples de uma interface que contempla o contexto do projeto.A estrutura é composta por uma entrada de texto e um elemento em tela para visualização das respostas geradas, exemplificado na figura 3.1.

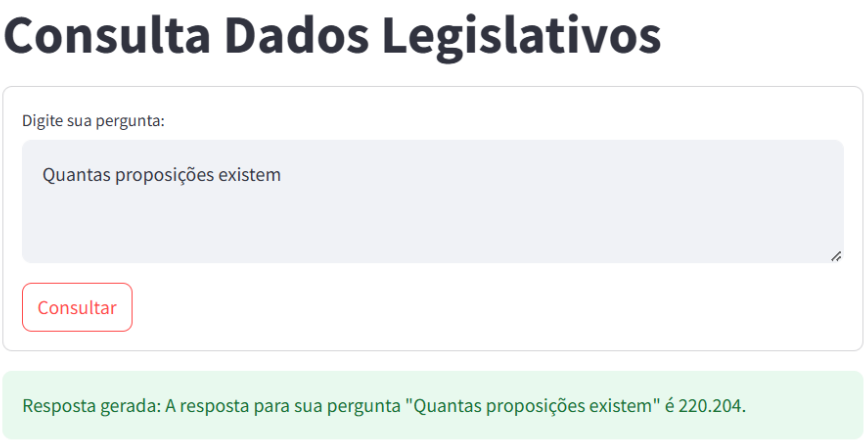


Figura 3.1: Interface em Streamlit

A execução final do projeto está descrita no código 3.13, que contém a configuração inicial da página em Streamlit e o formulário a ser utilizado, já na execução da consulta é feita a execução dos códigos descritos anteriormente em LangChain, mas com alguns tratamentos para uma melhor estrutura do projeto.

```
st.title("Consulta Dados Legislativos")

with st.form("form"):
    user_input = st.text_area("Digite sua pergunta:",
       placeholder="Exemplo: Quantos autores existem?")
    submitted = st.form_submit_button("Consultar")

if submitted:
    if not os.environ["OPENAI_API_KEY"].startswith("sk-"):
        st.error("Chave de API da OpenAI invalida!")
    else:
        with st.spinner("Consultando..."):
            try:
                response = "Erro: Nao foi possivel gerar uma
                   resposta."
                # Executar o grafo e obter apenas a ultima
                   resposta
                state = {"question": user_input}
                final_state = None  # Variavel para armazenar o
                   estado final

                # Iterar sobre os passos do grafo
                for step in graph.stream(state, stream_mode="
                   updates"):
                    final_state = step  # Atualiza o estado
                       final a cada passo

```

```
22                  # Acessar a resposta final
23                  if final_state and "generate_answer" in
                        final_state:
24                      response = final_state['generate_answer']['
                            answer']
25
26
27                  st.success(f"Resposta gerada: {response}")
28              except Exception as e:
29                  st.error(f"Erro ao processar a consulta: {e}")
```

Código 3.13: Função que gera a interface em Streamlit

3.5 Testes e Validação

Após a implementação do sistema, foram realizados testes para validar o funcionamento do sistema. Diversas consultas foram feitas, abrangendo diferentes tipos de proposições, anos, status e autores, para validar a estrutura criada no banco de dados e o uso correto.

Os resultados das consultas geradas pela LLM foram comparados com a base de dados original para garantir a precisão das informações retornadas, como mostra figura 3.2 que retorna a mesma informação da pergunta feita na figura 3.1.

Para fazer a validação das respostas é possível fazer consultas diretas ao banco de dados. Para isso, as validações podem ser feitas com respostas mais simples e esperadas, como quantidades e tipo dos dados exibidos, por exemplo, se trabalharmos com a quantidade total de proposições, podemos executar uma consulta direto a base dados e verificar se o sistema retornou a mesma informação.

```
mysql> SELECT COUNT(*) FROM dados_abertos.Proposicoes;
+----------+
| COUNT(*) |
+----------+
|   220204 |
+----------+
1 row in set (0.01 sec)

mysql>
```

Figura 3.2: Resposta da Consulta Feita ao Banco de Dados. Fonte: Própio autor

Seguindo com as validações, no contexto da base de dados é possível solicitar ao sistema por exemplo: “qual o autor com mais proposições?”, como mostra a consulta da figura 3.3.

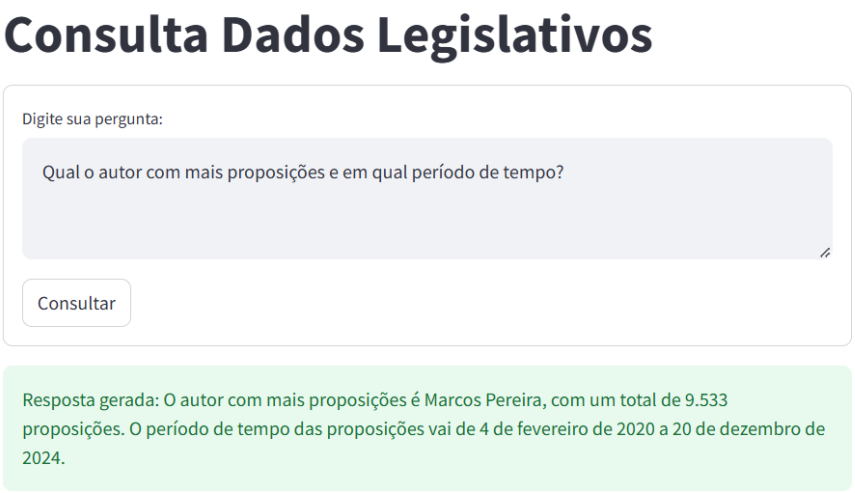


Figura 3.3: Figura 3.3: Consulta do autor com mais proposições. Fonte: Próprio autor

Agora, montando uma consulta SQL com base na pergunta e executando a mesma consulta gerada na base de dados local de dados abertos temos, conforme o resultado na figura 3.4, a mesma resposta gerada pela LLM, porém com o dado em sua forma mais bruta.

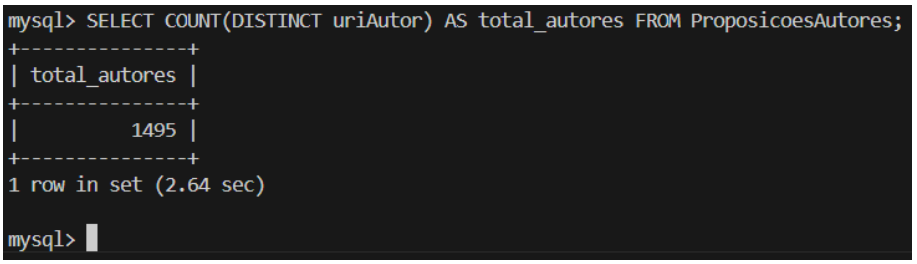
```
mysql> SELECT nomeAutor, COUNT(idProposicao) AS totalProposicoes, MIN(dataApresentacao) AS periodoInicio, MAX(dataApresentacao) AS periodoFim FROM
ProposicoesAutores JOIN Proposicoes ON ProposicoesAutores.idProposicao = Proposicoes.id GROUP BY nomeAutor ORDER BY totalProposicoes DESC LIMIT 1;
+---------------+------------------+---------------------+---------------------+
| nomeAutor     | totalProposicoes | periodoInicio       | periodoFim          |
+---------------+------------------+---------------------+---------------------+
| Marcos Pereira |             9533 | 2020-02-04 21:50:09 | 2024-12-20 17:48:00 |
+---------------+------------------+---------------------+---------------------+
1 row in set (2.37 sec)

mysql>
```

Figura 3.4: Consulta do autor com mais proposições. Fonte: Própio autor.

Seguindo essa mesma estrutura, é possível executar consultas mais simples, apenas para validar a conexão do sistema com a base de dados. Pode-se testar, por exemplo, a quantidade de itens nas tabelas, fazendo essa consulta pela interface e em seguida executando uma consulta direta ao banco de dados. Fazendo a pergunta pela interface de quantos autores existem, conforme demonstra a figura 3.5.

Figura 3.5: Consulta do autor com mais proposições. Fonte: Própio autor.

A resposta anterior obtida, tem que ser a mesma, em termo dos dados, que a consulta feita diretamente na base de dados, como mostra a figura 3.6.



```
mysql> SELECT COUNT(DISTINCT uriAutor) AS total_autores FROM ProposicoesAutores;
+---------------+
| total_autores |
+---------------+
|          1495 |
+---------------+
1 row in set (2.64 sec)

mysql>
```

Figura 3.6: Consulta do autor com mais proposições. Fonte: Própio autor.

Dessa forma é possível comprovar que o texto gerado pela LLM está seguindo uma estrutura amigável, simples e compreensível. Assim como as consultas geradas pelo modelo são coerentes com o contexto das perguntas e corretas, demonstrando também o sucesso de conexão entre a base e o sistema.

Capítulo 4

Resultados

Nesta seção, os resultados deste projeto são apresentados, sendo organizado em três subseções principais. A seção 4.1 tem como objetivo apresentar a interface criada, a 4.2 para analisar o desempenho do sistema em relação a coerência das consultas geradas e a quantidade de tokens necessários para cada execução e para finalizar a seção 4.3 para exemplificar casos de uso do sistema.

Além disso, são discutidos os resultados relacionados a usabilidade, desempenho do sistema e integração dos componentes. Os exemplos práticos e os testes realizados ilustram a capacidade do sistema de atender aos objetivos estabelecidos, promovendo acessibilidade e eficiência na consulta de dados legislativos.

4.1 Interface

Um dos objetivos do projeto é fornecer ao usuário uma experiência simples e prática, em que o seu uso seja intuitivo e acessível, dessa forma a interface se divide em duas funcionalidades a de entrada de texto, para que o usuário seja capaz de fazer suas solicitações e uma forma de visualizar as respostas geradas pelo sistema.

Dessa forma, é possível ter uma disposição simples e intuitiva dos elementos, o campo de texto destacado em uma estrutura de entrada de dados, acompanhado de um botão que executa a consulta e em seguida por uma visualização logo abaixo da resposta gerada pelo sistema.

A figura 4.1 demonstra como está o funcionamento da interface, mas especificamente como ela se comporta em uma tela de resolução comum de notebook, 1440 x 720 pixels.

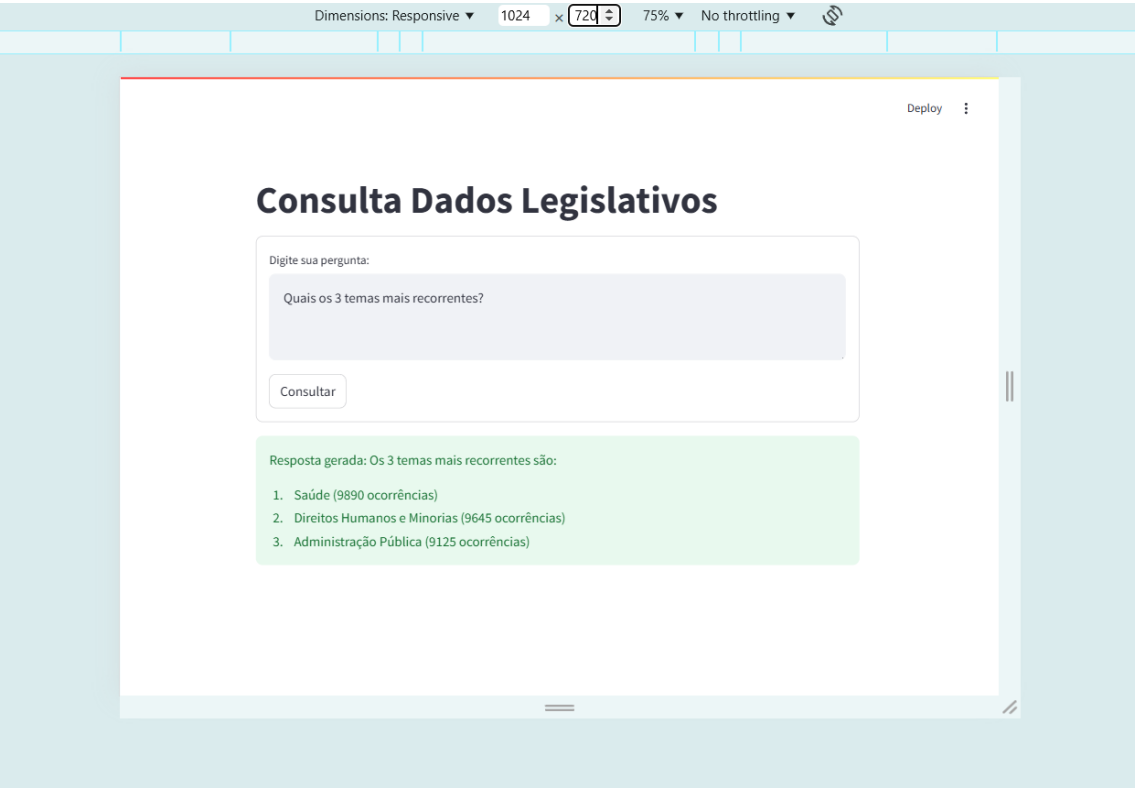


Figura 4.1: Visualização da interface em uma tela de notebook. Fonte: Própio autor.

Existem casos em que o usuário queira acessar o sistema utilizando outro dispositivo, com a biblioteca do Streamlit a responsividade é um quesito atendido, pois permite que haja uma visualização do sistema em telas menores, garantindo uma responsividade ao projeto, como mostrado na figura 4.2.

Figura 4.2: Visualização da interface em uma tela de dispositivo móvel. Fonte: Própio autor.

As respostas do sistema ficam em destaque, sendo atualizadas de acordo com a consulta feita pelo usuário, assim como um destaque com uma cor mais chamativa indicando o sucesso da consulta e a resposta a gerada.

4.2 Desempenho

Existem algumas formas de consultas que podem ser elaboradas para serem executadas pelo sistema, que possibilita testar a capacidade de compreensão do modelo. Perguntas com interpretação textual podem ser feitas, utilizando termos que não são referências diretas a forma como os dados estão organizados, como não usar o termo "tema", para se referir a educação, o resultado mostrado pode ser visualizado na figura 4.3.

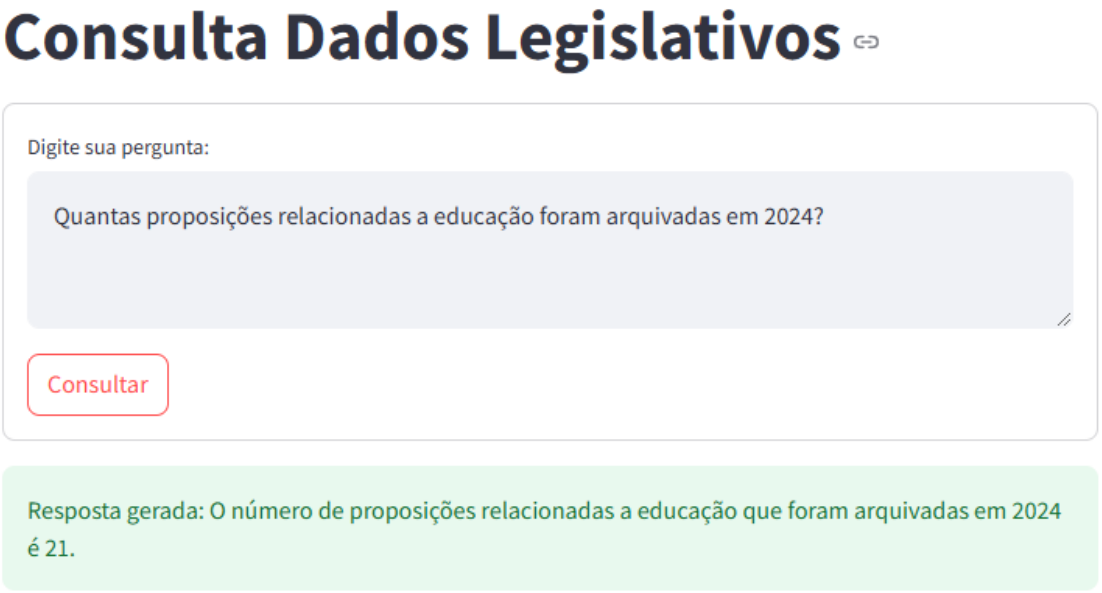


Figura 4.3: Resposta do sistema à uma pergunta "Quantas proposições relacionadas ao tema educação foram arquivadas em 2024?". Fonte: Própio autor.

4.2.1 Interpretação Contextual

Perguntas que exigem um pouco mais de interpretação, não foram um problema para a LLM, como teste foi executada a consulta "Quantos projetos de lei relacionadas a educação estão arquivadas?", mesmo sem usar termos comuns como proposição e tema, o modelo conseguiu retornar uma resposta coerente, como mostra a figura 4.4.

Figura 4.4: Resposta do sistema à uma pergunta com interpretação textual. Fonte: Própio autor.

Coletando os dados de execução, é possível visualizar a quantidade de tokens gerados, que a consulta teve, e a resposta efetiva que conseguiu criar em resposta a solicitação, como mostra a tabela 4.1.

Tabela 4.1: Resultado da interaçãoo

Query Gerada	`SELECT p.id, p.uri, p.siglaTipo, p.numero, p.ano, u.descricaoSituacao FROM Proposicoes p JOIN UltimoStatus u ON p.id = u.proposicaoId JOIN ProposicoesTemas pt ON p.uri = pt.uriProposicao WHERE pt.tema LIKE 'educação' AND u.descricaoSituacao LIKE 'arquivada' AND p.ano = 2024 ORDER BY p.dataApresentacao DESC LIMIT 1000;`
Tokens Utilizados	Entrada: 2,358 tokens, Saída: 305 tokens, Total: 2,663 tokens.

Com este resultado é possível analisar que a entrada teve um uso considerado de tokens em relação a saída, isso ocorre devido a resposta do banco ter sido menor do que a da entrada criada pelo usuário, não sendo necessário dividir o texto em vários tokens para criação da resposta.

4.2.2 Reconhecimento de Termos Ambíguos

Uma outra abordagem de verificação da LLM é utilizar termos ambíguos, que podem ser sinônimos, mas são na verdade maneiras de escrever determinados termos chave. Para validar a utilização de ambiguidade, foi utilizado o termo "Alterações Constitucionais"e

desse modo o sistema conseguiu retornar uma resposta coerente para a pergunta, como mostra a figura 4.5.

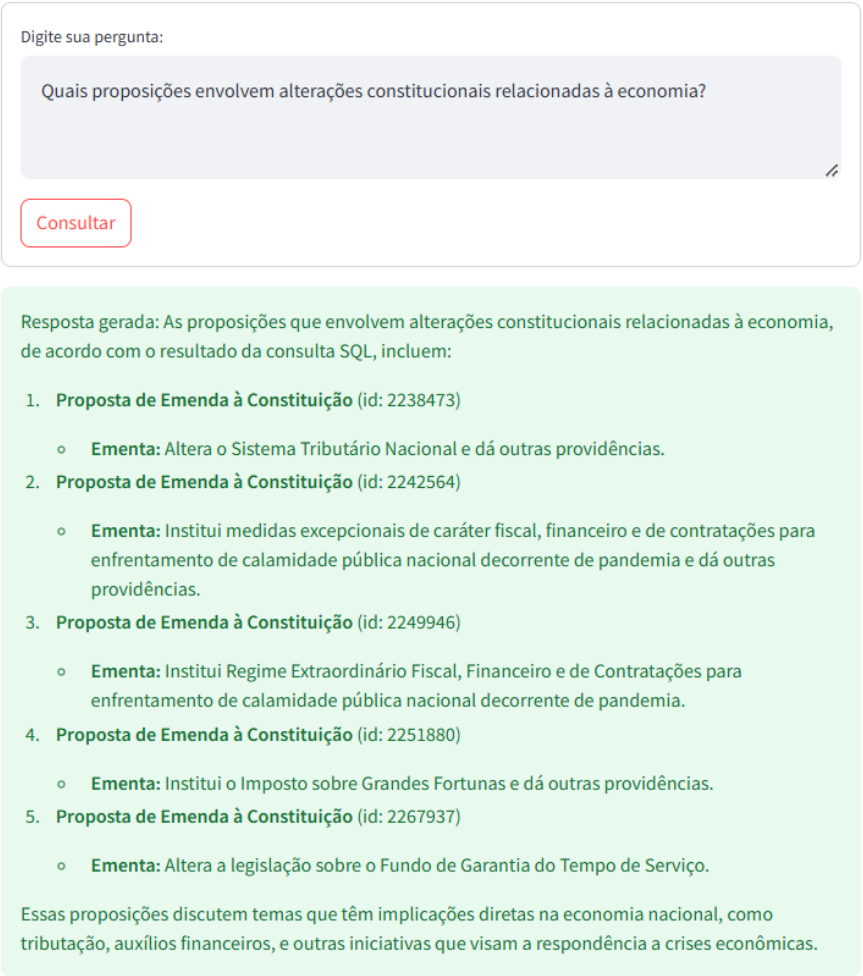


Figura 4.5: Resposta do sistema à uma pergunta com termo ambíguo. Fonte: Própio autor.

Como é possível ver na figura 4.5, a entrada e saída de tokens foi parecida, o que indica uma certa consistência do modelo e é interessante a forma como ele gera as consultas com base nas condições solicitadas, como `Economia`

Tabela 4.2: Resultado da interação

Query Gerada	`SELECT p.id, p.uri, p.ementa FROM Proposicoes p JOIN ProposicoesTemas pt ON p.uri = pt.uriProposicao WHERE pt.tema LIKE 'Economia' AND p.ementa LIKE 'constitucional' ORDER BY p.dataApresentacao DESC LIMIT 1000;`
Tokens Utilizados	Entrada: 2,295 tokens, Saída: 1,337 tokens, Total: 3,632 tokens.

4.2.3 Falsas Consultas

Para testar a compreensão do modelo, é possível utilizar termos que não condizem com o contexto ou inserir palavras equivocadas, como por exemplo "sdafsdafasdfdas", que deve ser incompreensível a primeira instante pois dificulta análise, mas o sistema pode ser capaz de gerar uma resposta capaz de explicar ao usuário a situação, como indica a figura 4.6.

Figura 4.6: Resposta do sistema a uma consulta errada. Fonte: Própio autor.

Um outro fator que influencia no uso dos tokens, é o retorno desta consulta ao banco de dados, que pode gerar uma resposta extensa, exigindo que o modelo tenha um uso mais elevado de tokens, para poder interpretar todo o conteúdo, como mostra os resultados da tabela 4.3.

Tabela 4.3: Resultado da interação

Query Gerada	`SELECT p.id, p.uri, pa.nomeAutor, pa.siglaPartidoAutor, pa.ordemAssinatura FROM Proposicoes p JOIN ProposicoesAutores pa ON p.id = pa.idProposicao LIMIT 1000;`
Tokens Utilizados	Entrada: 2,287 tokens, Saída: 40,069 tokens, Total: 42,3565 tokens.

4.3 Casos de Uso

O uso dessa ferramenta pode ser bastante diversa, pois ela pode ser útil em realizar consultas simples na base de dados, ou em consultas mais elaboradas, como por exemplo listas e textos mais estruturados.

Existem alguns cenários que o sistema pode ter um bom destaque e mostrar sua utilidade, as informações que ele retorna como resposta, podem ser utilizadas para complementar apresentações, para servir como fonte para um dado em uma reportagem ou em um artigo, pois ele é uma consulta direta a base de dados e pode ser uma fonte confiável de informação.

A principal vantagem em utilizar o sistema é a praticidade que ela proporciona ao estar conectado a base de dados e poder gerar respostas rápidas para perguntas feitas, funcionando como um assistente para consulta de informações sobre dados legislativos.

Outro exemplo interessante é utilizar o sistema para fazer consultas que permitem utilizar desses dados para fazer uma análise, como por exemplo consultar quais os temas mais populares de proposições de acordo com o ano, como mostra a figura 4.7, essa resposta sendo utilizada em análise com a situação do Brasil no período explicam o motivo para aquele tema estar em alta e é uma ótima forma para fazer uma análise temporal do país.

Figura 4.7: Resposta do sistema à uma pergunta.

Capítulo 5

Conclusão

Este trabalho tem como principal objetivo desenvolver uma solução que facilite o acesso aos dados disponibilizados pela da Câmara dos Deputados, de uma forma prática e acessível. De maneira que o usuário não precise utilizar de conhecimentos técnicos para realizar consultas à base de dados e obter respostas. A solução desenvolvida foi feita por meio do uso de um modelo de linguagem de grande escala, que se comunica com os dados e com o usuário.

Com o desenvolvimento do sistema e os resultados obtidos, é válido analisar que contempla ao que foi proposto. A sua estrutura de conexão, entre base dados, modelo de linguagem e usuário, proporcionam um funcionamento claro e bastante funcional, que se aproveita da qualidade de cada uma das ferramentas utilizadas.

Isso ocorre por conta da maneira de armazenamento, que é uma forma estruturada, bem definida de armazenar dados, o uso de uma LLM capaz de interpretar textos em linguagem natural, e comunicar com o banco de dados e gerar uma resposta coerente com o contexto da solicitação e os dados. Assim como o uso de uma interface simples e funcional, que entrega ao usuário uma boa experiência para fazer o uso dessa consulta de dados.

A performance do modelo de linguagem se mostrou bastante crucial para o funcionamento do projeto e como a escolha do modelo se conecta com o contexto proposto. Nos vários casos de execução listados foi possível ver um bom entendimento do modelo para as entradas, assim como um uso similar de tokens, possibilitando gerar consultas corretas que atendem a solicitação. E principalmente na geração das respostas, que mesmo exigindo um pouco mais do uso de tokens, dependendo do resultado da consulta ao banco, conseguiu processar e retornar uma resposta coerente.

Apesar de atender as expectativas, existem melhorias que podem ser implementadas, que podem criar um sistema ainda mais completo. Alguns usuários podem querer uma interação mais completa com os dados, como por exemplo poder gerar gráficos e tabelas pelo chat e uma integração com uma biblioteca como Pandas poderia ser interessante para que o usuário tenha essa possibilidade.

Os Prompts podem sempre ser aprimorados para adequarem ainda mais as respostas aos usuários. Uma integração mais elaborada poderia ser o desenvolvimento de um sistema de autenticação, para poder ter um controle maior de acesso e maior segurança para o sistema.

Referências Bibliográficas

BOMMASANI, R.; DAVIS, E.; ET AL. (2022), On the opportunities and risks of foundation models., Página na internet.
URL: *https://arxiv.org/abs/2108.07258*

BROWN, T.; MANN, B.; RYDER N.; et al. (2020*a*), Language models are few-shot learners., OMG Specification formal/2020-22-07.
URL: *https://arxiv.org/abs/2005.14165*

BROWN, T.; MANN, B.; RYDER N.; et al. (2020*b*), Language models are few-shot learners., OMG Specification formal/2020-22-07.
URL: *https://arxiv.org/abs/2005.14165*

CHEN, P. P (1976*a*), *The Entity-Relationship Model: Toward a Unified View of Data.*, 1ª edição, ACM Transactions on Database Systems.

CHEN, P. P (1976*b*), *The Entity-Relationship Model: Toward a Unified View of Data.*, 1ª edição, ACM Transactions on Database Systems.

Codd, E. F. (1970), *A Relational Model of Data for Large Shared Data Banks.*, 1ª edição, Communications of the ACM.

Dados Abertos (2025), Página na internet, Câmara dos Deputados.
URL: *https://dadosabertos.camara.leg.br/*

Docker (2025), Docker mysql, Página na internet, Docker.
URL: *https://hub.docker.com/_/mysql/*

Elmasri, R., & Navathe S. B. (2016), *Fundamentals of Database Systems.*, 7ª edição, Pearson.

HARRISON, Richard. (2023), *Building Applications with LangChain: Leveraging Large Language Models in Real-World Scenarios.*, 1ª edição, O'Reilly Media.

IDEAS, SOFTWARE (2025), Chen er diagram - entity-relationship diagram in chen notation, Página na internet.
URL: *https://www.softwareideas.net/chen-er-diagram-erd.*

Introdução ao JSON (2025), Página na internet, JSON ORG.
URL: *https://www.json.org/json-pt.html*

Langchain (2025*a*), Build a question/answering system over sql data, Página na internet, Langchain.
URL: `https://python.langchain.com/docs/tutorials/sql_qa/`

Langchain (2025*b*), Documentation: Sql database chain., Página na internet, Langchain.
URL: `https://python.langchain.com/docs/modules/chains/popular/sql_database/`

LIU, P.; YU, X.; JIANG J. (2023), *Prompt Engineering for Large Language Models: Best Practices and Applications.*, 1ª edição, Springer.

MANNING, Christopher D.; SCHÜTZE, Hinrich. (1999), *Foundations of statistical natural language processing.*, 1ª edição, MIT Press, Cambridge, MA.

OpenAI (2024), Gpt-4o mini: advancing cost-efficient intelligence, Página na internet, OpenAI.
URL: `https://openai.com/`

Oracle (2025), Mysql, Página na internet, Oracle.
URL: `https://www.mysql.com/`

RUSSELL, S.; NORVIG, P. (2021), *Inteligência Artificial: Estruturas e Estratégias para Solução Complexa de Problemas.*, 3ª edição, Pearson, São Paulo.

SHNEIDERMAN, B.; PLAIN, C.; COHEN M. (2016), *Designing the User Interface: Strategies for Effective Human-Computer Interaction.*, 6ª edição, Pearson, Boston.

Streamlit (2025), Build an llm app using langchain, Página na internet, Streamlit.
URL: `https://streamlit.io/`

Tuychiev, Bex (2024), How to set up and configure mysql in docker, Página na internet, Datacamp.
URL: `https://www.datacamp.com/tutorial/set-up-and-configure-mysql-in-docker`

(Docker 2025) (Tuychiev 2024) (*Dados Abertos* 2025) (Langchain 2025*a*) (Langchain 2025*b*) (Streamlit 2025) (Oracle 2025) (*Introdução ao JSON* 2025) (OpenAI 2024) (Codd 1970) (Elmasri 2016) (CHEN 1976*a*) (CHEN 1976*b*) (IDEAS 2025) (RUSSELL 2021) (MANNING 1999) (LIU 2023) (HARRISON 2023) (BOMMASANI 2022) (BROWN 2020*a*) (SHNEIDERMAN 2016) (BROWN 2020*b*)

Apêndice A

Informações adicionais

A.1 Código em Python para preencher a tabela de proposições

A seguir, o trecho código responsável pela lógica:

```
# Iterar sobre as proposicoes no JSON
for proposicao in proposicoes_data["dados"]:
    # Inserir na tabela Proposicoes
    cursor.execute("""
        INSERT INTO Proposicoes (
            id, uri, siglaTipo, numero, ano, codTipo,
               descricaoTipo, ementa,
            dataApresentacao
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
    """, (
        proposicao["id"], proposicao["uri"], proposicao["
           siglaTipo"],
        proposicao["numero"], proposicao["ano"], proposicao["
           codTipo"],
        proposicao["descricaoTipo"], proposicao["ementa"],
        proposicao["dataApresentacao"]
    ))


    # Obter o ID da proposicao recem-inserida
    proposicao_id = proposicao["id"]

    # Inserir na tabela UltimoStatus
    ultimo_status = proposicao["ultimoStatus"]
    cursor.execute("""
        INSERT INTO UltimoStatus (
            proposicao_id, data, sequencia, uriRelator, codOrgao
               , siglaOrgao,
            uriOrgao, regime, descricaoTramitacao,
```

```
                idTipoTramitacao,
            descricaoSituacao, idSituacao, despacho, apreciacao, url
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    """, (
        proposicao_id, ultimo_status["data"], ultimo_status["sequencia"],
        ultimo_status["uriRelator"], ultimo_status["codOrgao"], ultimo_status["siglaOrgao"],
        ultimo_status["uriOrgao"], ultimo_status["regime"],
        ultimo_status["descricaoTramitacao"], ultimo_status["idTipoTramitacao"],
        ultimo_status["descricaoSituacao"], ultimo_status["idSituacao"],
        ultimo_status["despacho"], ultimo_status["apreciacao"], ultimo_status["url"]
    ))
```

Código A.1: Exemplo de código em Python

A.2 Código em Python para preencher a tabela de Autores de Proposições

A seguir, apresentamos um exemplo de código para ilustrar o uso do LaTeX:

```

# Iterar sobre os autores no JSON
for autor in autores_data["dados"]:
    # Converter ordemAssinatura para inteiro
    ordem_assinatura = int(autor["ordemAssinatura"])
    # Converter ordemAssinatura para booleano
    proponente = autor["proponente"] == "1"
    nome_autor = autor["nomeAutor"][:255]

    # Inserir na tabela AutoresProposicoes
    cursor.execute("""
        INSERT INTO ProposicoesAutores (
            idProposicao, uriProposicao, uriAutor, codTipoAutor, tipoAutor,
            nomeAutor, siglaPartidoAutor, uriPartidoAutor, siglaUFAutor,
            ordemAssinatura, proponente
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    """, (
```

```
        autor["idProposicao"], autor["uriProposicao"], autor["uriAutor"],
        autor["codTipoAutor"], autor["tipoAutor"], nome_autor,
        autor["siglaPartidoAutor"], autor["uriPartidoAutor"], autor["siglaUFAutor"],
        ordem_assinatura, proponente
    ))
```

Código A.2: Exemplo de código em Python

A.3 Código em Python para preencher a tabela de Temas de Proposições

A seguir, apresentamos um exemplo de código para ilustrar o uso do LaTeX:

```

# Iterar sobre as proposicoes no JSON
for proposicao in temas_data["dados"]:
    # Inserir na tabela ProposicoesTemas
    if "tema" in proposicao and "codTema" in proposicao:
        cursor.execute("""
            INSERT INTO ProposicoesTemas (
                uriProposicao, siglaTipo, numero, ano, codTema, tema, relevancia
            ) VALUES (%s, %s, %s, %s, %s, %s, %s)
        """, (
            proposicao["uriProposicao"], proposicao["siglaTipo"], proposicao["numero"],
            proposicao["ano"], proposicao["codTema"], proposicao["tema"],
            proposicao.get("relevancia", 0)  # Use 0 como valor padrao para relevancia
        ))
```

Código A.3: Exemplo de código em Python

A.4 Código em Python para execução do sistema

A seguir, apresentamos um exemplo de código para ilustrar o uso do LaTeX:

```
import streamlit as st
from langchain_openai.chat_models import ChatOpenAI
from langchain_community.utilities import SQLDatabase
from langchain_openai import ChatOpenAI
from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool
```

```
import os
from typing_extensions import TypedDict
from langchain import hub
from typing_extensions import Annotated
from langgraph.graph import START, StateGraph

os.environ["OPENAI_API_KEY"] = "Inserir sua chave"
os.environ["LANGCHAIN_API_KEY"] = "Inserir sua chave"
os.environ["LANGSMITH_TRACING"] = "true"


class State(TypedDict):
    question: str
    query: str
    result: str
    answer: str


mysql_uri = 'mysql+mysqlconnector://root:<senha>@localhost:<
    porta_banco>/<database>'

db = SQLDatabase.from_uri(mysql_uri)

llm = ChatOpenAI(model="gpt-4o-mini")

query_prompt_template = hub.pull("langchain-ai/sql-query-system-
    prompt")

assert len(query_prompt_template.messages) == 1
query_prompt_template.messages[0].pretty_print()



class QueryOutput(TypedDict):
    """Generated SQL query."""

    query: Annotated[str, ..., "Syntactically valid SQL query."]


def write_query(state: State):
    """Generate SQL query to fetch information."""
    prompt = query_prompt_template.invoke(
        {
            "dialect": db.dialect,
            "top_k": 1000,
            "table_info": db.get_table_info(),
            "input": state["question"],
```

```
        }
    )
    structured_llm = llm.with_structured_output(QueryOutput)
    result = structured_llm.invoke(prompt)
    return {"query": result["query"]}

def execute_query(state: State):
    """Execute SQL query."""
    execute_query_tool = QuerySQLDataBaseTool(db=db)
    return {"result": execute_query_tool.invoke(state["query"])}

def generate_answer(state: State):
    """Answer question using retrieved information as context.
       """
    prompt = (
        "Given the following user question, corresponding SQL
           query, "
        "and SQL result, answer the user question.\n\n"
        f'Question: {state["question"]}\n'
        f'SQL Query: {state["query"]}\n'
        f'SQL Result: {state["result"]}'
    )
    response = llm.invoke(prompt)
    return {"answer": response.content}


graph_builder = StateGraph(State).add_sequence(
    [write_query, execute_query, generate_answer]
)

graph_builder.add_edge(START, "write_query")
graph = graph_builder.compile()


# Interface do Streamlit
st.title("Consulta Inteligente ao Banco de Dados")

# Formulario para entrada de texto
with st.form("form"):
    user_input = st.text_area("Digite sua pergunta:",
       placeholder="Exemplo: Quantos autores existem?")
    submitted = st.form_submit_button("Consultar")

# Processamento da consulta
if submitted:
    if not os.environ["OPENAI_API_KEY"].startswith("sk-"):
        st.error("Chave de API da OpenAI invalida!")
```

```
    else:
        with st.spinner("Consultando..."):
            try:
                response = "Erro: Nao foi possivel gerar uma resposta."
                # Executar o grafo e obter apenas a ultima resposta
                state = {"question": user_input}
                final_state = None  # Variavel para armazenar o estado final

                # Iterar sobre os passos do grafo
                for step in graph.stream(state, stream_mode="updates"):
                    final_state = step  # Atualiza o estado final a cada passo

                # Acessar a resposta final
                if final_state and "generate_answer" in final_state:
                    response = final_state['generate_answer']['answer']


                st.success(f"Resposta gerada: {response}")
            except Exception as e:
                st.error(f"Erro ao processar a consulta: {e}")
```

Código A.4: Exemplo de código em Python